

GroupFunctions.jl: computing individual entries of the irreducible representations of the unitary group $U(d)$

David Amaro-Alcalá¹ and Konrad Szymański¹

¹ Research Center for Quantum Information, Institute of Physics, Slovak Academy of Sciences, Dúbravská cesta 9, Bratislava, Slovakia

DOI: [N/A](#)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: 01 January 1970

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

[GroupFunctions.jl](#)¹ is a Julia library for computing individual matrix entries of irreducible representations of $U(d)$. These entries, called group functions, can be evaluated symbolically or numerically. For $SU(2)$, they reduce to the Wigner D -functions. The library computes these matrix entries in a carrier-space basis enumerated by Gelfand-Tsetlin patterns ([Gelfand & Tsetlin, 1950](#)). It can also compute entire representation operators, construct input unitaries from parameterisations common in quantum optics, translate Gelfand-Tsetlin patterns into occupation-number kets, and compute the associated Schur functions.

Statement of need

Representations of the unitary group $U(d)$ arise in many subfields of physics and mathematics, and computations often reduce to evaluating their matrix entries, called group functions. For an irreducible representation labelled by λ , the corresponding object is – in the mathematicians' and physicists' inner product notation – the following:

$$D_{\text{out},\text{init}}^{(\lambda)}(U) = (D^{(\lambda)}(U)\text{init}, \text{out}) = \langle \text{out} | D^{(\lambda)}(U) | \text{init} \rangle,$$

where init and out denote basis vectors in the representation carrier space.

In mathematics, summing the diagonal group functions gives the trace of the representation matrix of U . This trace is the character of the representation and the Schur polynomial of the eigenvalues of U , an object central to algebraic combinatorics and symmetric function theory.

In quantum physics, group functions appear in several settings. In quantum optics, a group function gives the transition amplitude of photons through a linear optical network ([de Guise et al., 2014](#)). The same object can be used to characterise the performance of quantum devices ([Amaro-Alcalá, 2026](#)) and to describe the symmetry properties of states ([Otto & Szymański, 2024](#)). One important subproblem is boson sampling ([Aaronson & Arkhipov, 2011](#)), where the transition amplitude reduces to a permanent whose evaluation is classically computationally hard. Whether noisy real-world quantum devices can perform this computation remains an open question.

Some of these tasks require only a numerical estimate, whereas others require an exact symbolic group function. [GroupFunctions.jl](#) addresses the latter. Although related software exists (see below), to our knowledge none is designed to compute a single representation-matrix

¹David Amaro-Alcalá developed the package, its algorithms, and its test suite, and prepared the initial documentation. Konrad Szymański substantially revised and expanded the documentation and developed additional examples, and contributed to the standardisation of the API.

entry as an exact symbolic D -function. We therefore compare the cost of computing one such entry with the conventional route that constructs the Lie-algebra representation matrices and exponentiates the resulting full matrix.

The cost of forming the full matrix depends on the dimension $D_\lambda(d)$ of the representation, the number of rows and columns in $D^{(\lambda)}(U)$. Fix a Young diagram λ with $N > 1$ boxes and at most d rows. The basis vectors are indexed by semistandard Young tableaux of this shape with entries in $\{1, \dots, d\}$. The hook-content formula counts these tableaux (Fulton, 1997, sec. 4.3, p. 55). Equivalently, the dimension is the Schur polynomial at d arguments equal to 1, because the character at the identity equals the dimension:

$$D_\lambda(d) = s_\lambda(\underbrace{1, \dots, 1}_d) = \prod_{(i,j) \in \lambda} \frac{d + j - i}{h_{ij}}.$$

Here (i, j) denotes a box in row i and column j , its content is $j - i$, and its hook length h_{ij} counts the box itself, those to its right in the same row, and those below it in the same column. For fixed λ , the product has N factors linear in d and a constant denominator, so $D_\lambda(d) = \Theta(d^N)$. For example, $\lambda = (2)$ gives $D_{(2)}(d) = d(d + 1)/2$.

To compute one entry directly, we keep the two tableau labels fixed as d grows and supply the corresponding basis vectors, avoiding enumeration of the full basis. In this setting, the current symbolic implementation has time complexity $\Theta(d^3)$ and space complexity $\Theta(d^2)$. The cubic term comes from constructing the tables that index the monomial sum: assembling each length- d row recursively copies successively longer arrays, costing $\Theta(d^2)$ operations per row across d rows. With N fixed, the number of tables and permutations remains bounded as d grows. These estimates use a unit-cost model for array access, copying individual entries, and scalar arithmetic, with space measured by the maximum number of scalar entries stored simultaneously. The dependence on N is absorbed into the constants; bit complexity is excluded.

For comparison, suppose the $D_\lambda(d) \times D_\lambda(d)$ Lie-algebra matrix has already been constructed. Numerical scaling and squaring with a Padé approximant uses dense matrix products and a linear solve (Higham, 2005). With the Padé degree and number of squarings fixed, standard dense multiplication and elimination require $\Theta(D_\lambda(d)^3) = \Theta(d^{3N})$ scalar arithmetic operations and have space complexity $\Theta(D_\lambda(d)^2) = \Theta(d^{2N})$. Here each scalar addition, subtraction, multiplication, or division has unit cost; in a numerical implementation, these operations use fixed-precision arithmetic. For $\lambda = (2)$, these costs grow as d^6 and d^4 , respectively, compared with d^3 and d^2 for the direct calculation. The numerical route produces the full matrix at a chosen U ; the direct route produces one exact polynomial in its entries. This comparison explains the cost of forming the full matrix when only one entry is needed. It applies with N fixed and does not compare against numerical methods designed to compute individual entries.

State of the field

Several existing packages relate to GroupFunctions.jl but address different problems. SUNRepresentations.jl (QuantumKitHub contributors, 2020) and the algorithm presented in the appendix of (Alex et al., 2011) compute $SU(d)$ Clebsch-Gordan coefficients. RepLAB (Rosset et al., 2021) supports manipulating irreducible representations of various groups, including $U(d)$, but provides only indirect numerical access to group functions. IntegrateUnitary.jl (Pawela & Puchała, 2026) performs symbolic integration over compact groups rather than evaluating representation matrices. haarpy (Cardin et al., 2024) implements Weingarten-calculus methods. Other libraries focus on quantum optics. BosonSampling.jl (Seron & Restivo, 2024), Perceval (Heurtel et al., 2023), and Q0ptCraft (Aguado et al., 2023) numerically model linear optical devices, while The Walrus (Gupt et al., 2019) helps compute amplitudes for Gaussian boson sampling. These packages do not target the symbolic

computation of individual representation-matrix entries, which is the primary purpose of `GroupFunctions.jl`.

Software design

This library provides a unified method for computing representation matrix entries of $U(d)$ irreducible representations specified by integer partitions of length at most d , including computations with symbolic input matrices. Several computational routes are possible in principle. For symmetric irreducible representations, which model fully indistinguishable bosons, one can manipulate states as polynomials of creation operators applied to the vacuum. Another approach constructs and exponentiates the Lie algebra generators in the chosen representation. Both approaches are computationally expensive. More restricted methods based on generating functions also exist (Prakash, 1996).

The authors chose the Grabmeier-Kerber formula (Grabmeier & Kerber, 1987) as the most general solution. It expresses the entry of a matrix as a sum of monomials in the entries of the input matrix, weighted by the irreducible representation and the basis vectors in question. Our implementation optimises the enumeration of the double cosets that index this sum by grouping permutations that contribute the same monomial. This implementation provides the library's main function, `group_function`.

Internally, the algorithm represents basis vectors as semistandard Young tableaux. The user-facing functions expose the equivalent Gelfand-Tsetlin patterns through the `GTPattern` data structure and provide utility functions for common quantum optics scenarios, such as `occupation_number`.

Research impact statement

`GroupFunctions.jl` has been used to evaluate matrix entries and group characters in published work on filtered randomized benchmarking (Amaro-Alcalá, 2026). The library has been under continuous development since 2020 and is tested in CI; it is registered in the Julia General registry under the MIT licence, installable with the following command:

```
] add GroupFunctions
```

The following example evaluates, symbolically, an entry of a matrix between basis vectors of the $U(4)$ mixed-symmetry irreducible representation.

```
λ = [2,2,1,0]; basis=basis_states(λ); # integer partition and basis
group_function(λ, basis[1], basis[end]) # symbolic matrix entry
```

The result is a polynomial in the entries of the $U(4)$ matrix. For symmetric irreducible representations the entry of a matrix reduces to a permanent, which symbolic algebra packages also compute; for mixed-symmetry ones no existing software computes these entries symbolically. Further examples, applications, and mathematical background are available in the documentation.

AI usage disclosure

OpenAI Codex (GPT-5.3) assisted with optimising the performance of the double-coset enumeration at a late stage. The authors developed the mathematical design and proof of correctness of the optimised algorithm. Anthropic Claude (Opus 4.8) assisted with code review and language review of the documentation and manuscript. The authors reviewed and edited all AI-assisted changes.

Acknowledgements

We thank Dr. Hubert de Guise for helpful discussions and suggestions on the bibliography, and Dr. Alonso Botero for suggestions that improved the presentation. Mitacs CALAREO, DeQHOST APVV-22-0570, QUAS VEGA 2/0164/25, Postdokgrant APD0161, and the Štefan Schwarz programme supported this work. David Amaro-Alcalá acknowledges the indirect support of the Government of Alberta and NSERC during his PhD studies at the University of Calgary.

References

- Aaronson, S., & Arkhipov, A. (2011). The computational complexity of linear optics. *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*, 333–342. <https://doi.org/10.1145/1993636.1993682>
- Aguado, D. G., Gimeno, V., Moyano-Fernández, J. J., & Garcia-Escartin, J. C. (2023). QOptCraft: A Python package for the design and study of linear optical quantum systems. *Computer Physics Communications*. <https://doi.org/10.1016/j.cpc.2022.108511>
- Alex, A., Kalus, M., Huckleberry, A., & Delft, J. von. (2011). A numerical algorithm for the explicit calculation of $SU(N)$ and $SL(N, \mathbb{C})$ Clebsch-Gordan coefficients. *Journal of Mathematical Physics*, 52(2), 023507. <https://doi.org/10.1063/1.3521562>
- Amaro-Alcalá, D. (2026). Kostant relation in filtered randomized benchmarking for passive bosonic devices. In *Physical Review A* (Vol. 113). American Physical Society. <https://doi.org/10.1103/xmlq-vssg>
- Cardin, Y., de Guise, H., & Quesada, N. (2024). Haarp, a Python library for Weingarten calculus and integration of classical compact groups and ensembles. In *GitHub repository* (0.0.6). <https://github.com/polyquantique/haarp>; GitHub.
- de Guise, H., Tan, S.-H., Poulin, I. P., & Sanders, B. C. (2014). Coincidence landscapes for three-channel linear optical networks. *Physical Review A*, 89(6). <https://doi.org/10.1103/physreva.89.063819>
- Fulton, W. (1997). *Young tableaux: With applications to representation theory and geometry* (Vol. 35). Cambridge University Press. <https://doi.org/10.1017/CBO9780511626241>
- Gelfand, I. M., & Tsetlin, M. L. (1950). Finite-dimensional representations of the group of unimodular matrices. *Doklady Akademii Nauk SSSR*, 71, 825–828.
- Grabmeier, J., & Kerber, A. (1987). The evaluation of irreducible polynomial representations of the general linear groups and of the unitary groups over fields of characteristic 0. *Acta Applicandae Mathematicae*, 8(3), 271–291. <https://doi.org/10.1007/bf00046717>
- Gupt, B., Izaac, J., & Quesada, N. (2019). The Walrus: A library for the calculation of hafnians, Hermite polynomials and Gaussian boson sampling. *Journal of Open Source Software*, 4(44), 1705. <https://doi.org/10.21105/joss.01705>
- Heurtel, N., Fyrrillas, A., Gliniasty, G. de, & others. (2023). Perceval: A software platform for discrete variable photonic quantum computing. *Quantum*, 7, 931. <https://doi.org/10.22331/q-2023-02-21-931>
- Higham, N. J. (2005). The scaling and squaring method for the matrix exponential revisited. *SIAM Journal on Matrix Analysis and Applications*, 26(4), 1179–1193. <https://doi.org/10.1137/04061101X>
- Otto, F., & Szymański, K. (2024). Rotational covariance restricts available quantum states. *Physical Review A*, 110(2), 022440. <https://doi.org/10.1103/PhysRevA.110.022440>

- Pawela, Ł., & Puchała, Z. (2026). *IntegrateUnitary.jl: A Julia package for symbolic integration over Haar measures*. <https://arxiv.org/abs/2605.23830>
- Prakash, J. (1996). Wigner's D-matrix elements for SU(3)-a generating function approach. *Journal of Physics A: Mathematical and General*, 29(18), 6059–6072. <https://doi.org/10.1088/0305-4470/29/18/032>
- QuantumKitHub contributors. (2020). *SUNRepresentations.jl*. <https://github.com/QuantumKitHub/SUNRepresentations.jl>.
- Rosset, D., Montealegre-Mora, F., & Bancal, J.-D. (2021). *RepLAB: A computational/numerical approach to representation theory*. <https://github.com/replab/replab>; Springer International Publishing. https://doi.org/10.1007/978-3-030-55777-5_60
- Seron, B., & Restivo, A. (2024). BosonSampling.jl: A Julia package for quantum multi-photon interferometry. *Quantum*, 8, 1378. <https://doi.org/10.22331/q-2024-06-18-1378>